

1. JSX

JSX is the syntax commonly used inside React components. It looks similar to HTML, but it is actually written inside JavaScript code. React uses JSX to describe what the UI should look like.

Basic JSX

```
function Welcome() {  
  return <h1>Hello React</h1>;  
}
```

The important idea is that the component returns a UI description. JSX can contain normal HTML-like elements, but JavaScript expressions are placed inside `{ }`.

```
const name = "Denny";  
  
function Welcome() {  
  return <h1>Hello, {name}</h1>;  
}
```

Important JSX rules

- Use **className** instead of HTML's **class**.
- JavaScript expressions go inside `{ }`.
- A component normally returns one root element, or use a fragment `<>...</>`.
- JSX attributes use JavaScript-style names such as **onClick**.
- Components must begin with an uppercase letter, such as **ProductCard**.

```
function Product() {  
  const price = 100;  
  
  return (  
    <div className="product">  
      <h2>Keyboard</h2>  
      <p>Price: ${price}</p>  
    </div>  
  );  
}
```

JSX Exercise

Exercise 1: Create a **Profile** component with variables for name, age, and favorite game. Display all three values using JSX expressions.

Exercise 2: Create a **Product** component with name, price, and stock variables. Display the product information in a card-like div.

Challenge: Add a discount variable and calculate the final price before displaying it.

2. Components

A React application is normally divided into components. A component is a JavaScript function that returns UI. The purpose is to split a large page into smaller, understandable pieces.

```
function Header() {  
  return <header>My Website</header>;  
}  
  
function App() {  
  return (  
    <>  
      <Header />  
      <main>  
        <h1>Home</h1>  
      </main>  
    </>  
  );  
}
```

```
        </main>
      </>
    );
  }
```

The **<Header />** syntax tells React to render the Header component. Components can be imported from other files using normal JavaScript import/export.

```
export default function Header() {
  return <header>My Website</header>;
}

// App.jsx
import Header from "../Header";
```

Good component thinking

- Use a component when a piece of UI has a clear purpose.
- Reusable components should avoid depending on one specific page whenever possible.
- Do not make every tiny HTML element its own component just because you can.
- A useful component can later receive data through props.

Component Exercises

Exercise 3: Split a simple homepage into **Header**, **Navbar**, **MainContent**, and **Footer** components.

Exercise 4: Create three components called **ProductCard**, **ProductList**, and **App**. For now, hard-code the product information.

3. useState

Normal JavaScript variables do not automatically cause React to render the UI again when they change. `useState` gives a component a value that React remembers between renders and provides a function for changing that value.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Add
      </button>
    </div>
  );
}
```

In `const [count, setCount] = useState(0)`: `count` is the current state value, `setCount` changes the state, and `0` is the initial value.

Why not just use a variable?

```
let count = 0;

function add() {
  count++;
}
```

Changing this variable does not tell React that the UI needs to be rendered again. For UI data that changes over time, use state.

State with objects and arrays

```
const [user, setUser] = useState({
  name: "Alex",
  score: 10
});

setUser({
  ...user,
  score: user.score + 1
});
```

When changing an object or array in React state, create a new object/array rather than directly changing the old one. The spread operator is commonly used for this.

useState Exercises

Exercise 5: Make a counter with +1, -1, and Reset buttons.

Exercise 6: Make a button that switches between "ON" and "OFF".

Exercise 7: Create a shopping list. A button should add a new item to an array in state.

Exercise 8: Create a score object containing player name and score. Add a button that increases the score.

Challenge: Create a simple inventory with items and quantities. Add buttons that increase or decrease an item's quantity.

4. useEffect

useEffect is used when a component needs to perform a side effect. A side effect is work that happens because of rendering but is not simply calculating the UI.

Common examples

- Fetching data from an API
- Changing the browser document title
- Starting a timer
- Subscribing to an external event
- Running code when particular state/props values change

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    document.title = "My React App";
  }, []);

  return <h1>Hello</h1>;
}
```

The empty dependency array [] means this effect runs after the initial render. If dependencies are provided, React can run the effect again when those values change.

```
useEffect(() => {
  console.log("Search changed:", search);
}, [search]);
```

Cleanup

Some effects create something that should later be stopped or removed. In those cases, return a cleanup function.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("tick");
  }, 1000);

  return () => {
    clearInterval(timer);
  };
}, []);
```

Common useEffect mistakes

- Using useEffect for calculations that could simply be done during rendering.
- Leaving out dependencies when the effect uses changing values.
- Creating timers or event listeners without cleanup.
- Putting every piece of application logic inside useEffect.

useEffect Exercises

Exercise 9: Change the browser tab title whenever a counter changes.

Exercise 10: Create a timer that increases every second. Add cleanup so the interval does not continue after the component is removed.

Exercise 11: Create a search state and useEffect that prints a message whenever the search text changes.

Challenge: Create a component that displays the current count and changes the document title to match it.

5. Foundation Challenge

Build a small **Game Scoreboard**. This exercise combines the main ideas from the two-week review.

- Create an App component and at least two smaller components.
- Store the player's score using `useState`.
- Have buttons for +1, -1, and Reset.
- Display the player's name and score.
- Use `useEffect` to change the browser title when the score changes.
- Keep the score from going below 0.
- Use CSS to make the scoreboard look like a small UI card.

Extra Challenge

- Add a second player.
- Add a winner message when a player reaches 10 points.
- Add a Reset Game button.
- Keep the player data in an object or array in state.
- Split the UI into reusable `PlayerCard` component.

Foundation Review Questions

1. What is JSX, and how is it different from normal HTML?
2. What is a React component?
3. Why do we split a React application into multiple components?
4. What does `useState` do?
5. Why should we use the state setter instead of directly changing the state variable?
6. How do you update an object stored in state?
7. How do you update an array stored in state?
8. What is `useEffect` used for?
9. What does the dependency array `[]` mean?
10. When would you need a cleanup function in `useEffect`?
11. Give one example of something that should not need `useEffect`.
12. Explain the difference between state and a normal JavaScript variable.

Next Topic: Props